

Introduction to Hardware Design Languages and Tools (CITHN10001)

WiSe 2526, Implementation of a RISC-V Processor

Lab Manual

Chair of Computer Architecture and Operating System, CAOS, Prof. Trinitis

1 Introduction

In this lab, students are expected to deepen their understanding of Hardware Description language, digital circuit design, and computer architecture by implementing a RISC-V processor from scratch. It should support a subset of RV32I instructions, as listed in table 1. The full RISC-V reference is generally available on the Internet, e.g. RISC-V Card from SFU¹.

The implementation is partitioned into multiple groups of design questions so that students can approach the target incrementally. It is recommended to read through this manual to be clear about specifications and requirements before implementation.

31	25 24	20 19	15 14 12 11	7 6	0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12:10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode	B-type
imm[31:12]				rd	opcode	U-type
imm[20 10:1 11 19:12]				rd	opcode	J-type

Figure 1: RISC-V Instruction Formats

class	Type	Instructions
Arithmetic and logical (A&L-R)	R	add, sub, xor, or, and, sll, srl, sra, slt, sltu
Arithmetic and logical with immediate (A&L-I)	I	addi, xori, ori, andi, slli, srli, srai, slti, sltiu
Store to memory	S	sw
Load from memory	I	lw
Branch	B	beq
Jump	J	jal
Upper Immediate	U	lui

Table 1: Instructions to be implemented

¹https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf

2 Guidance on Design and Implementation

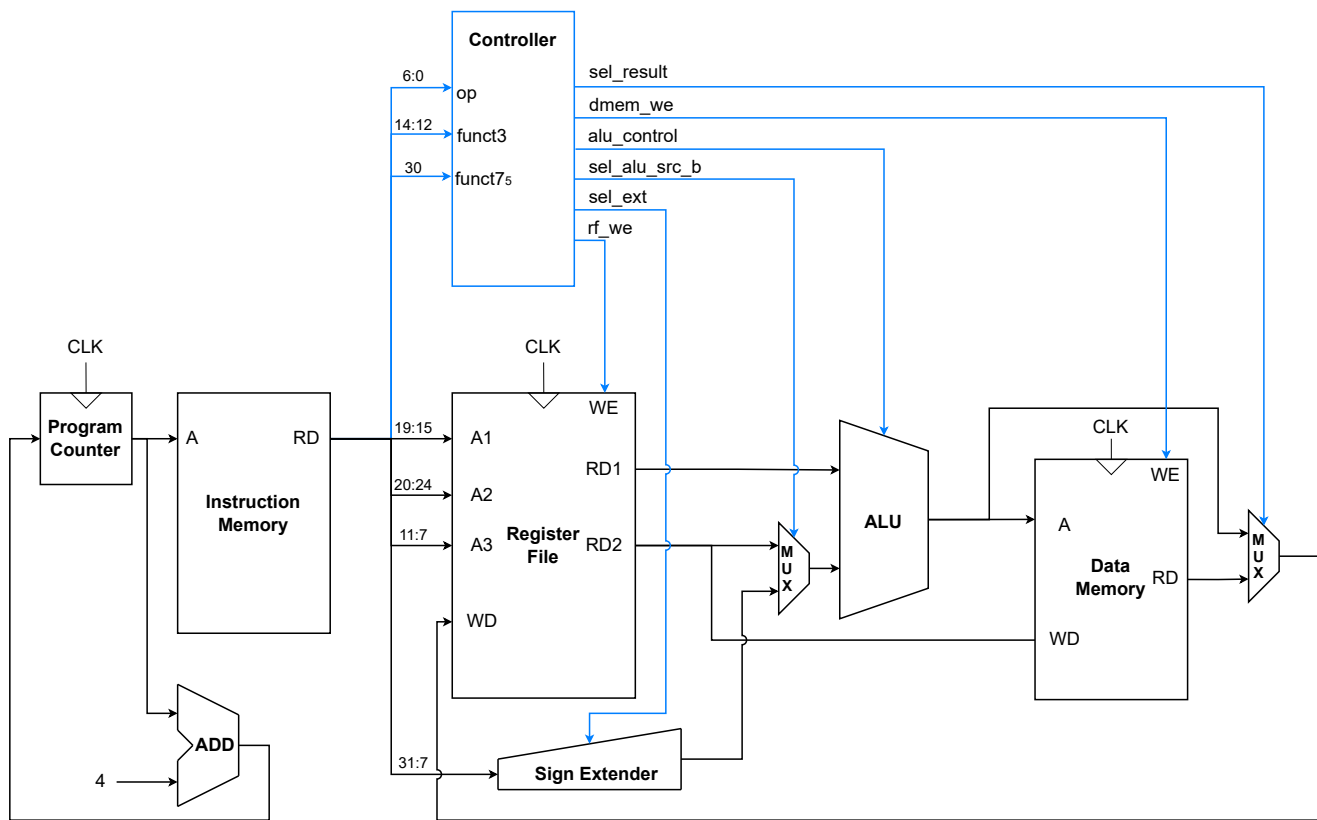


Figure 2: RISC-V processor: starting point

Figure 2 is the diagram of a basic system as the starting point. This system contains the necessary datapath for arithmetic and logical (both register and immediate) instructions, data memory load and store instruction, while branch, jump and the upper immediate manipulation can be added to it further.

2.1 Basic Modules

. It is recommended to start with the program counter, adder, multiplexer, register file, instruction memory and data memory, as they have standalone functionalities that are decoupled from the ISA.

- Consider the IO ports and then internal behavior of the module before implementation.
- Which modules use sequential logic, which use combinational logic?
- Which modules are better to be implemented with parameterization, to improve the module scalability?
- Do instruction memory and data memory follow byte addressing or word addressing?

2.2 Decoding and Execution

Design and implementation of sign extender, ALU and controller requires understanding of RISC-V ISA instruction format.

Sign extender simply extends the immediate value from different instruction types (see figure 1) to 32-bit.

- Which types store immediate values in the instruction so that that extension is needed?
- How to distinguish them and switch?
- How to re-arrange the bit for extension?

ALU is responsible for the arithmetic and logical operations, which is needed by arithmetic and logical instructions (R/I type). Nevertheless, data memory access instructions also require Add operation to compute addresses.

The behavior of ALU is to perform the correct operation on the input operands based on the input control signal. A multiplexer is needed to select between operations. Based on the R-type format, it is natural to use 3-bit **funct3** directly to distinguish the operations.

- Which Verilog syntax is suitable to build such multiplexing logic?
- add and sub has exactly the same **funct3**, how to further tell them by a minimal add-on? Also for srl and sra.
- How to use the same datapath for both add and sub? Pre-processing the operand may be needed.
- Both slt and sltu compares two values, what is the difference?
- Is it possible to use the add/sub datapath also for slt? overflow may be considered.
- Both srl and sra do the shift right, what is the difference?
- What does the table 2 look like for I-type arithmetic and logical operations? Can the Verilog code you have built also work for them?

Instruction	funct3	funct7
add	0x0	0x00
sub	0x0	0x20
sll	0x1	0x00
slt	0x2	0x00
sltu	0x3	0x00
xor	0x4	0x00
srl	0x5	0x00
sra	0x5	0x20
or	0x6	0x00
and	0x7	0x00

Table 2: R-type operations

Controller, also known as decoder or control unit, decodes the instruction fetched from the instruction memory, and then generates a valid set control signals to other components, like the write enable of data memory or selection of the multiplexer. That guarantees the system behaves as expected given a specific instruction.

To ensure the maintainability and readability, it is recommended to implement the decoding logic in two levels, by leveraging an intermediate signal **alu_op** that coarsely classify the instructions for ALU operations. Using **alu_op** and other signals, the final control signal to the ALU module can be generated.

Instruction(s)	opcode	rf_we	sel_ext	sel_alu_src_b	dmem_we	sel_result	alu_op
lw							00
sw							00
A&L (R-type)							01
A&L (I-type)							10

Table 3: Controller: first level decoding

alu_op	funct3	funct7	alu_control (operation)	target instruction(s)
00	-	-	{3'b000, 1'b0} (add)	lw/sw
01	000	7'h00		add
	000	7'h20		sub
	001	7'h20		sll
	...	...	...	...
10	000	-		addi
	001	-		slli
	...	...	...	...

Table 4: Controller: second level decoding

- Fill in two decoding tables of control signal values for A&L Instructions (R and I), data memory Load and Store instructions. More rows need to be added.
- Implement the two level decoding logic

2.3 System Integration

The full system can be assembled once all components are built, by module instantiation and connection. Clock and reset would be two necessary IO ports.

It is recommended to verify the system-level functionality with simple programs (see section 3) before implementing the BEQ, JAL, and LUI.

2.4 BEQ, JAL, and LUI

BEQ, JAL and LUI can be implemented on the top of existing system.

In general

- How to modify the decoding table to include three instructions?

BEQ

- Is it possible to implement BEQ by reusing add/sub operation in the ALU?
- By BEQ, next PC value can also come from the summation of current PC and immediate value. Should such summation be done in the ALU, if not, why? How to choose the source?
- What system components should be added to the system?

JAL

- How can the next PC be written into the register file?

LUI

- Where should the immediate value re-arrangement be done?

3 Guidance on Functional Verification

To perform functional verification of the processor, it is necessary to build a testbench module and prepare test program (binary).

3.1 Testbench

Testbench is the very top level module of the infrastructure. It is responsible for

- instantiation of the Design Under Test (DUT);
- generate the clock and reset signals for DUT;
- control the simulation procedure;
- control the waveform dump;
- initialize the instruction memory.

3.2 Test Programs

Functional verification ensures that simulation of the test program on the DUT behaves as expected.

- write a test program in RISC-V assembly which consists of implemented instructions;
- convert it to the binary, e.g. RISC-V online assembler²;
- load the binary file in testbench module. `readmemh` can initialize an array in the design hierarchy by provided binary file.

²<https://riscvasm.lucasteske.dev/>

4 Recommended Coding Style

- Except the testbench, all modules should be synthesizable Verilog.

5 Requirements and Deliverables

A mini report with

- Full system diagram (you can modify the source file of figure 2, by uploading `hdl-1-1.drawio` to drawio³)
- The complete decoding tables
- A RISC-V assembly program that you designed. It should use as much implemented instructions as possible.

³A free graph editor: <https://www.drawio.com/>